

# Wprowadzenie do sztucznej inteligencji

---

## Cwiczenie 02 - Algorytmy genetyczne i ewolucyjne

Raport z badan

Semestr letni, rok akademicki 2025/2026

Autor: Wiktor Sosnowski Numer albumu: 348 561

Warszawa, czerwiec 2026

---

## 1. Tresc zadania

Cel zadania polega na implementacji algorytmu genetycznego z mutacja, selekcja ruletkowa, krzyzowaniem jednopunktowym oraz sukcesja generacyjna. Zaimplementowany algorytm ma nastepnie posluzyc do optymalizacji klasycznego, symetrycznego problemu komiwojazera.

Kroki do wykonania:

1. Implementacja algorytmu genetycznego.
  2. Zastosowanie algorytmu do rozwiazania problemu komiwojazera. Eksperymentalne dobranie zestawu parametrow, dla ktorego algorytm daje dobry wynik.
  3. Zbadanie, w jaki sposob nastepujace zmiany wplyna na rezultaty osiagane przez algorytm:
    - zwiekszenie prawdopodobienstwa mutacji,
    - zmiana sposobu selekcji na turniejowa.
- 

## 2. Interpretacja zadania

Problem komiwojazera (TSP) polega na znalezieniu najkrotszej trasy odwiedzajacej kazde miasto dokladnie raz i wracajacej do punktu startowego. Jest to klasyczny problem optymalizacji kombinatorycznej klasy NP-trudnej.

Algorytm genetyczny nasladuje mechanizmy ewolucji biologicznej. Populacja osobnikow (permutacji miast) podlega kolejnym generacjom selekcji, krzyzowania i mutacji. Funkcja przystosowania jest odwrotnoscia dlugosci trasy - im krotsza trasa, tym wyzsze przystosowanie.

Selekcja ruletkowa przydziela kazdemu osobnikowi prawdopodobienstwo wyboru proporcjonalne do jego przystosowania. Selekcja turniejowa losuje k osobnikow i wybiera najlepszego - parametr k kontroluje sile presji selekcyjnej.

Krzyzowanie jednopunktowe dla permutacji: czesc trasy jednego rodzica (do losowo wybranego punktu) jest kopiowana do potomka, a pozostale miasta sa uzupełniane w kolejnosci wystepowania u drugiego rodzica, pomijajac juz umieszczone. Gwarantuje to poprawnosc permutacji.

Mutacja swap: z zadanyam prawdopodobienstwem zamieniane sa pozycje dwoch losowo wybranych miast w trasie.

Sukcesja generacyjna oznacza, że cała populacja jest zastępowana nowym pokoleniem potomków w każdej generacji.

---

### 3. Zbior danych

Użyto instancji berlin52 z biblioteki TSPLIB. Instancja zawiera 52 miasta o współrzędnych euklidesowych. Odległości obliczono jako odległości euklidesowe zaokrąglone do najbliższej liczby całkowitej, zgodnie ze standardem TSPLIB. Znane optymalne rozwiązanie wynosi 7542.

---

### 4. Opis implementacji

Implementacja składa się z dwóch modułów:

`tsp_utils.py` - wczytywanie plików TSPLIB, budowanie macierzy odległości, obliczanie długości trasy i funkcji przystosowania.

`genetic_algorithm.py` - klasa `GeneticAlgorithm` przyjmująca macierz odległości, rozmiar populacji, prawdopodobieństwo mutacji, liczbę generacji, metodę selekcji i rozmiar turnieju. Implementuje inicjalizację populacji losowymi permutacjami, selekcję ruletkową i turniejową, krzyżowanie jednopunktowe z naprawą permutacji, mutacje swap oraz sukcesję generacyjną.

Każdy wariant parametrów był uruchamiany 25 razy z różnym ziarnem generatora (seed 0-24). Wyniki zapisano do plików `results.json` i `raw_lengths.csv`.

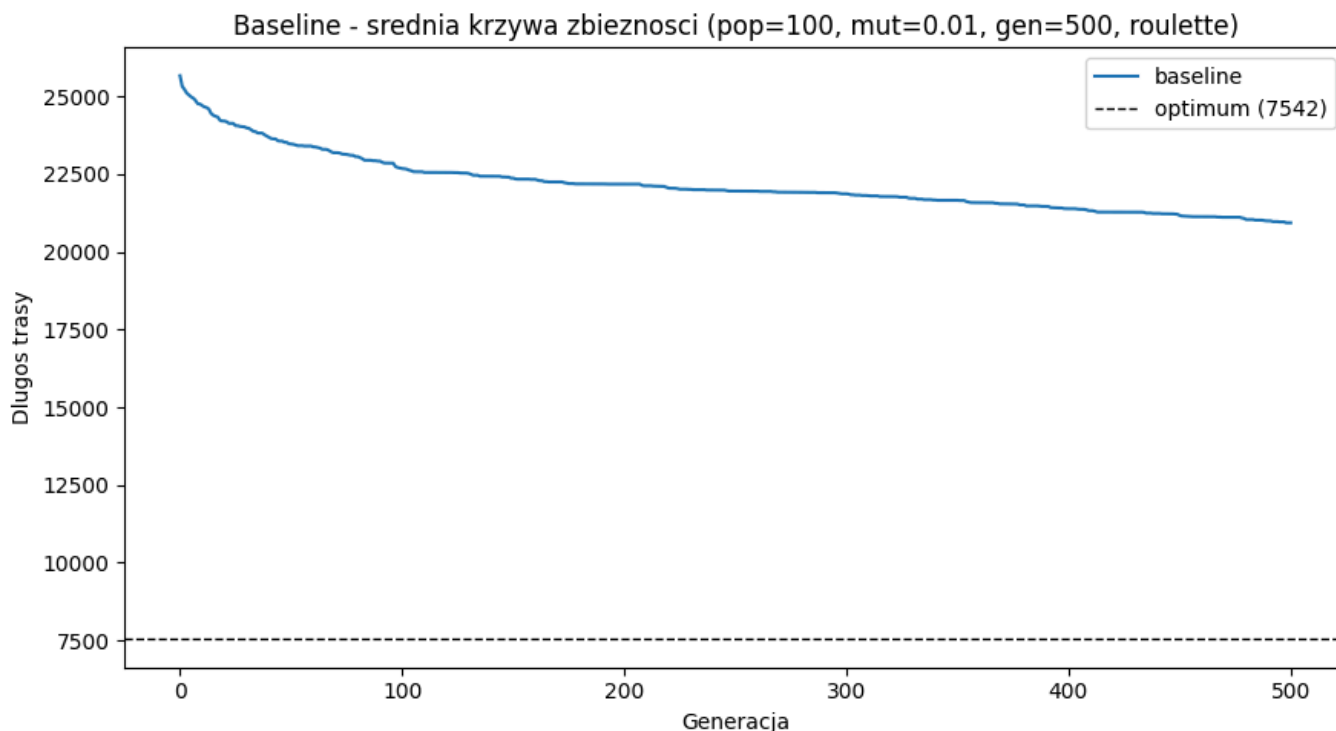
---

### 5. Konfiguracja bazowa (Baseline)

Parametry: `pop_size = 100`, `mutation_prob = 0,01`, `n_generations = 500`, `selection = roulette`.

| Wariant  | min      | srednia  | std     | max      |
|----------|----------|----------|---------|----------|
| baseline | 17876,00 | 20937,24 | 1048,06 | 22481,00 |

Uzyskane wyniki są znacznie powyżej optymalnego rozwiązania (7542), co jest oczekiwanym zachowaniem algorytmu bez elityzmu przy ograniczonej liczbie generacji. Wyniki stanowią punkt odniesienia dla kolejnych eksperymentów.

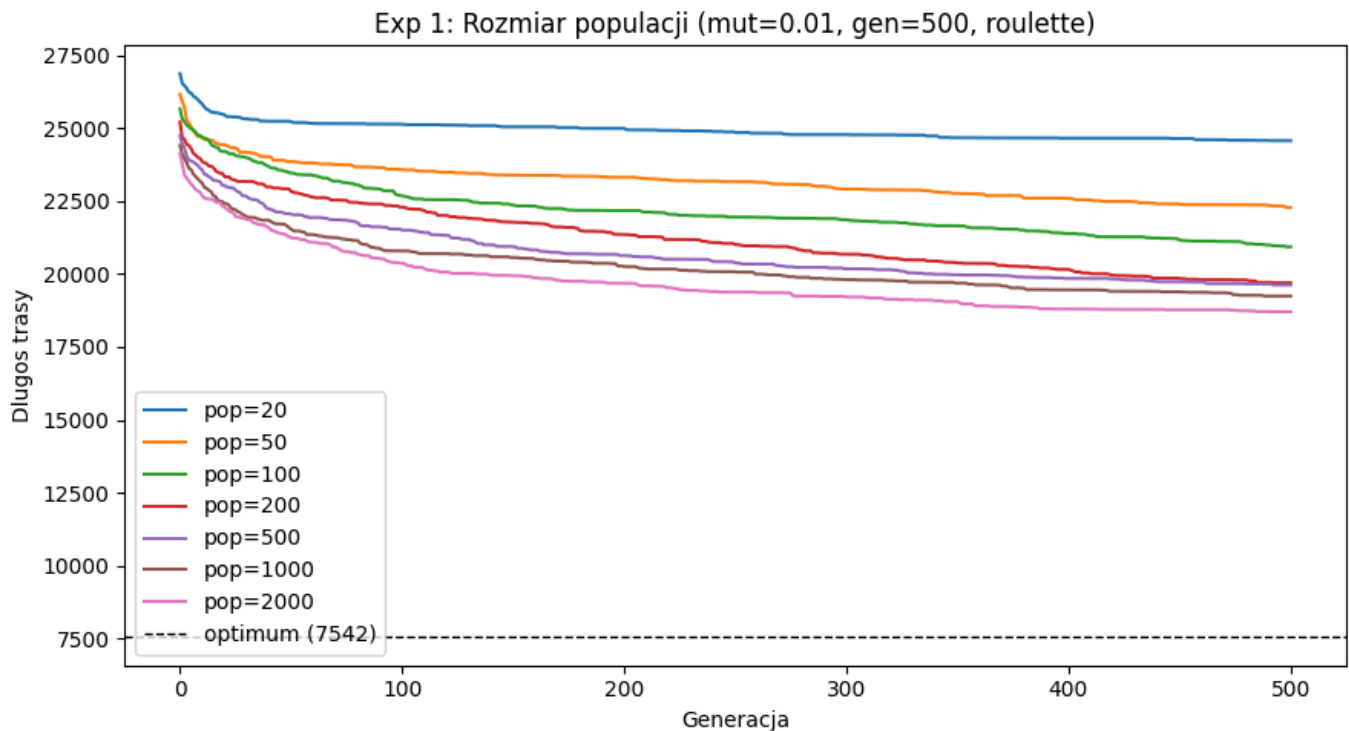


## 6. Eksperyment 1 - Rozmiar populacji

Stale parametry: mutation\_prob = 0,01, n\_generations = 500, selection = roulette. Badany zakres: pop\_size in {20, 50, 100, 200, 500, 1000, 2000}.

| Wariant         | min      | srednia  | std     | max      |
|-----------------|----------|----------|---------|----------|
| pop_size = 20   | 22158,00 | 24580,00 | 956,84  | 26262,00 |
| pop_size = 50   | 19776,00 | 22290,64 | 1080,27 | 25053,00 |
| pop_size = 100  | 17876,00 | 20937,24 | 1048,06 | 22481,00 |
| pop_size = 200  | 17963,00 | 19702,44 | 1001,07 | 21496,00 |
| pop_size = 500  | 16920,00 | 19629,24 | 920,07  | 21084,00 |
| pop_size = 1000 | 17693,00 | 19247,20 | 781,75  | 20612,00 |
| pop_size = 2000 | 15097,00 | 18710,24 | 1080,44 | 20118,00 |

Srednia dlugosc trasy maleje wraz ze wzrostem rozmiaru populacji - od 24580,00 dla pop\_size = 20 do 18710,24 dla pop\_size = 2000. Poprawa jest wyrazna w zakresie 20-200, natomiast od 500 wzwyż tempo poprawy spowalnia (roznica srednich miedzy pop\_size = 500 a 1000 wynosi 382, miedzy 1000 a 2000 wynosi 537). Najnizsze odchylenie standardowe osiagnieto dla pop\_size = 1000 (781,75). Przy pop\_size = 2000 odchylenie standardowe wzrasta do 1080,44 - porownywalne z mniejszymi populacjami - co wskazuje na niewystarczajaca liczbe generacji do zbiezności tak duzej populacji.



Wnioski: Większy rozmiar populacji poprawia średnią jakość wyników, jednak zysk maleje przy dużych wartościach. Wzrost odchylenia standardowego przy `pop_size = 2000` sugeruje, że przy stałej liczbie generacji (500) bardzo duże populacje nie zdążają się zbliżyć. Nie oznacza to, że duża populacja jest ogólnie niekorzystna - przy większej liczbie generacji efekt może być inny.

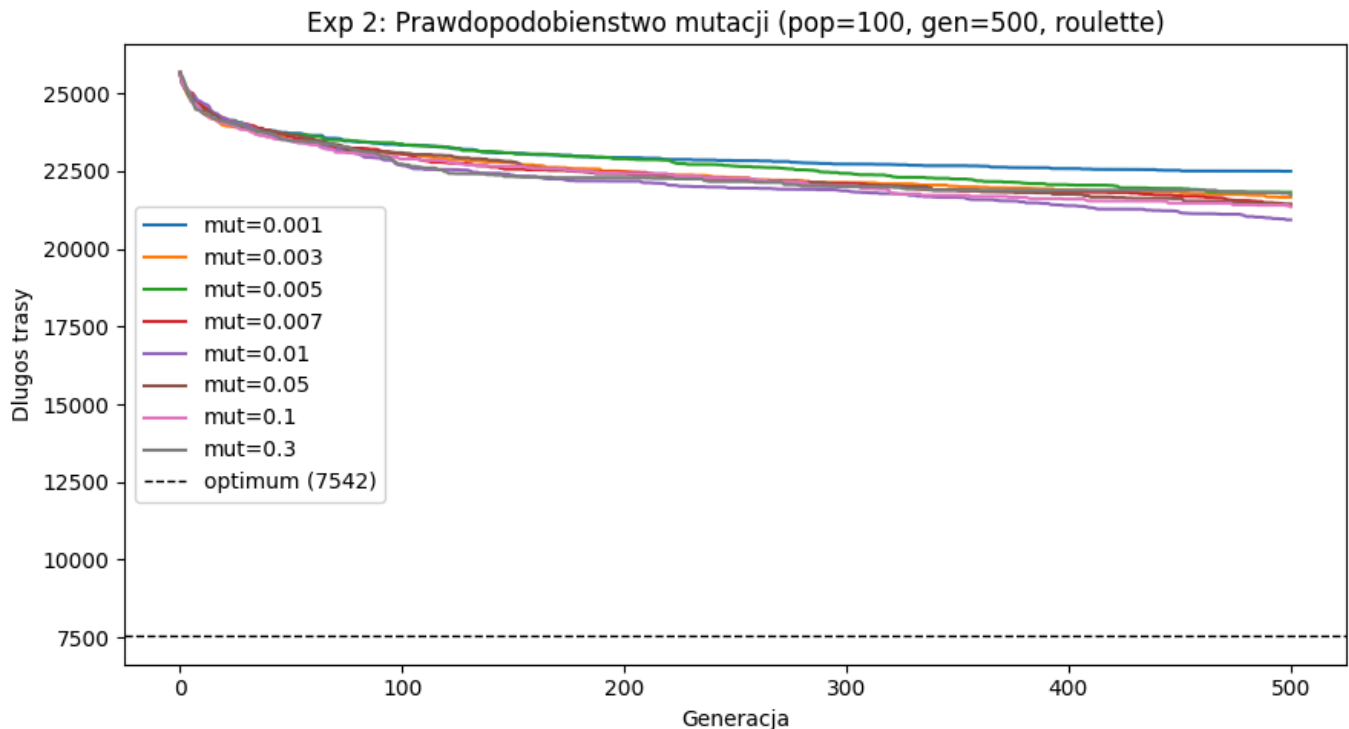
## 7. Eksperyment 2 - Prawdopodobieństwo mutacji

Stale parametry: `pop_size = 100`, `n_generations = 500`, `selection = roulette`. Badany zakres: `mutation_prob` in {0,001, 0,003, 0,005, 0,007, 0,01, 0,05, 0,1, 0,3}.

| Wariant                            | min      | średnia  | std     | max      |
|------------------------------------|----------|----------|---------|----------|
| <code>mutation_prob = 0,001</code> | 20871,00 | 22498,12 | 961,11  | 24610,00 |
| <code>mutation_prob = 0,003</code> | 19303,00 | 21660,84 | 1385,32 | 24203,00 |
| <code>mutation_prob = 0,005</code> | 18653,00 | 21823,20 | 1244,03 | 23962,00 |
| <code>mutation_prob = 0,007</code> | 18986,00 | 21403,84 | 1278,96 | 23646,00 |
| <code>mutation_prob = 0,01</code>  | 17876,00 | 20937,24 | 1048,06 | 22481,00 |
| <code>mutation_prob = 0,05</code>  | 20111,00 | 21446,00 | 799,25  | 23299,00 |
| <code>mutation_prob = 0,1</code>   | 19843,00 | 21354,32 | 821,29  | 22897,00 |
| <code>mutation_prob = 0,3</code>   | 19387,00 | 21738,32 | 750,77  | 23199,00 |

Najlepsza średnia długość trasy uzyskano dla `mutation_prob = 0,01` (20937,24). Niższe wartości dają gorsze wyniki - przy 0,001 średnia wynosi 22498,12, co wskazuje na zbyt słabą eksplorację przestrzeni. Powyżej 0,01 średnia rośnie i stabilizuje się w przedziale 21354-21823, co sugeruje, że zbyt częsta mutacja zaburza wzorce wypracowane przez krzyżowanie. Zależność nie jest monotoniczna - w przedziale 0,001-0,01 obserwujemy

nieregularny trend z lokalnym pogorszeniem przy 0,005, natomiast od 0,01 wzwyż srednie sa do siebie zblizone. Odchylenie standardowe maleje wraz ze wzrostem mutation\_prob (od 961 do 751), co oznacza wieksza stabilnosc wynikow kosztem gorszej srednicy.



Wnioski: Dla instancji berlin52 przy podanych parametrach bazowych optymalne prawdopodobienstwo mutacji wynosi 0,01. Zbyt mala mutacja ogranicza eksploracje, zbyt duza zaburza dziedziczenie dobrych cech. Znalezione optimum jest specyficzne dla tej konfiguracji - przy innych parametrach optymalna wartosc moze sie roznic.

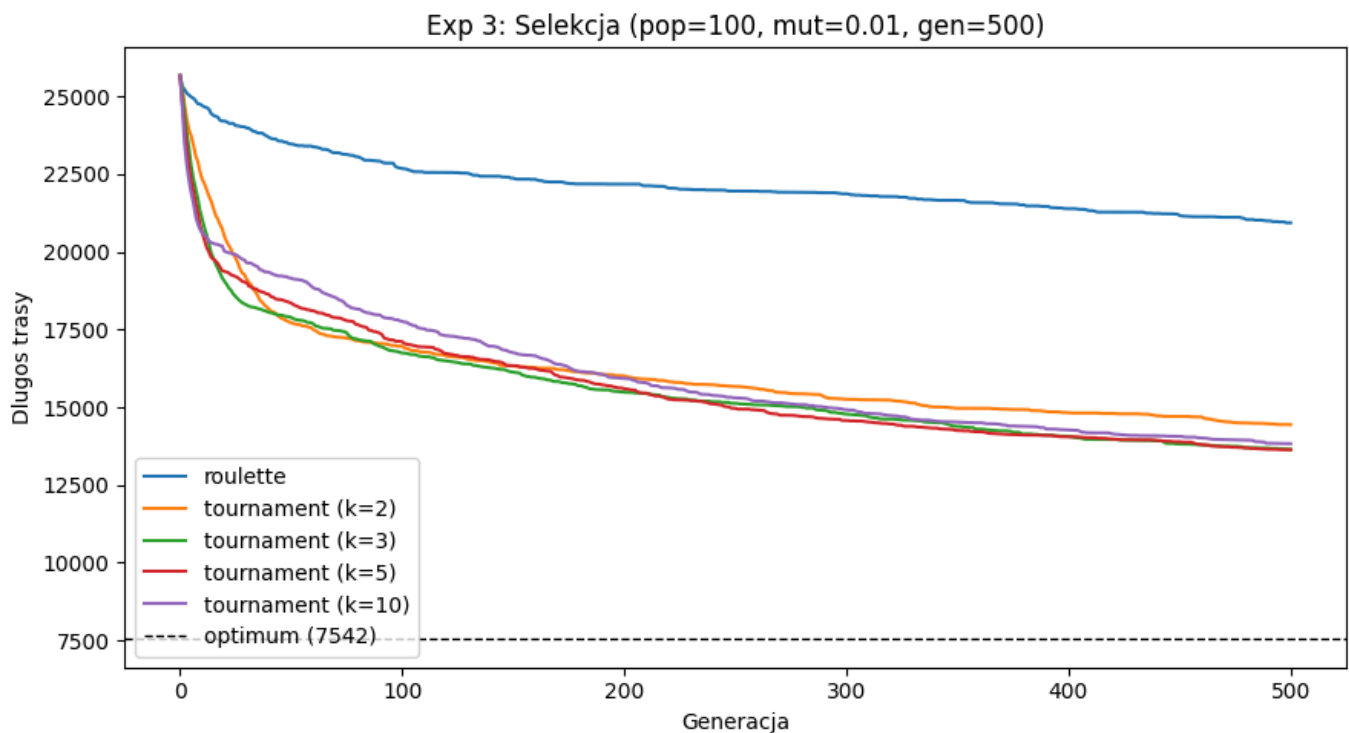
## 8. Eksperyment 3 - Metoda selekcji

Stale parametry: pop\_size = 100, mutation\_prob = 0,01, n\_generations = 500. Badane warianty: selekcja ruletkowa oraz selekcja turniejowa z rozmiarem turnieju k in {2, 3, 5, 10}.

| Wariant          | min      | srednia  | std     | max      |
|------------------|----------|----------|---------|----------|
| roulette         | 17876,00 | 20937,24 | 1048,06 | 22481,00 |
| turniej (k = 2)  | 13034,00 | 14443,84 | 949,16  | 16322,00 |
| turniej (k = 3)  | 12028,00 | 13655,16 | 860,98  | 15168,00 |
| turniej (k = 5)  | 11781,00 | 13632,20 | 792,04  | 15078,00 |
| turniej (k = 10) | 12447,00 | 13831,16 | 704,40  | 15659,00 |

Selekcja turniejowa przewyzsza ruletkowa we wszystkich badanych wariantach. Srednia dla roulette wynosi 20937,24, natomiast najlepszy wariant turniejowy (k = 5) osiaga srednia 13632,20 - poprawa o okolo 35%. Najlepszy wynik absolutny (min = 11781) rowniez nalezy do k = 5. Wsrod wariantow turniejowych najlepsza srednia osiagaja k = 5 (13632,20) i k = 3 (13655,16) - roznica jest niewielka. Przy k = 10 srednia rosnie do

13831,16, a minimum do 12447, co wskazuje na nadmierną presję selekcyjną i utratę różnorodności populacji. Odchylenie standardowe maleje monotonicznie wraz ze wzrostem  $k$  (od 949 do 704).



Wnioski: Selekcja turniejowa daje wyraźnie lepsze wyniki niż ruletkowa dla problemu komiwojagera w badanej konfiguracji. Optymalna wartość rozmiaru turnieju leży w okolicach  $k = 3-5$ . Zbyt mały turniej ( $k = 2$ ) nie wywiera wystarczającej presji selekcyjnej, zbyt duży ( $k = 10$ ) prowadzi do przedwczesnej zbieżności.

## 9. Wnioski końcowe

Najlepsze parametry uzyskane w eksperymentach (przy  $n_{\text{generations}} = 500$ ):

| Parametr        | Najlepsza badana wartość | Srednia dlugosci trasy |
|-----------------|--------------------------|------------------------|
| pop_size        | 2000                     | 18710,24               |
| mutation_prob   | 0,01                     | 20937,24               |
| metoda selekcji | turniej $k = 5$          | 13632,20               |

Najwyższy wpływ na jakość wyników miała zmiana metody selekcji z ruletkowej na turniejową - poprawa średniego wyniku o około 35% przy  $k = 5$ . Rozmiar populacji ma istotny wpływ, jednak zysk maleje przy dużych wartościach, a przy stałej liczbie generacji bardzo duże populacje wykazują wzrost wariancji. Prawdopodobieństwo mutacji ma umiarkowany wpływ z wyraźnym minimum w okolicach 0,01.

Wszystkie uzyskane wyniki są powyżej optymalnego rozwiązania instancji berlin52 (7542), co wynika z zastosowania sukcesji generacyjnej bez elityzmu oraz ograniczonej liczby generacji.